

SquadX.dev Spec

O módulo spec-native do SquadX.dev — desenvolvimento humano + IA

Resumo da proposta · rascunho para discussão com a equipe

Em uma frase

O SquadX.dev Spec é onde a **especificação vira a unidade de trabalho**: discutimos e versionamos a spec, ela gera as tarefas, humanos e agentes de IA executam lado a lado, e nada é dado como pronto sem passar por uma validação que confere o código contra a spec.

O SquadX.dev é uma plataforma mais ampla; este documento trata especificamente do SquadX.dev Spec — a parte de especificação e execução do trabalho.

O problema que estamos resolvendo

Hoje o trabalho se fragmenta entre ferramentas e o contexto se perde no caminho:

- A discussão e a spec vivem no chat; as tarefas no Jira; o código no repositório — três lugares desconectados.
- O “todo” se perde: é preciso reconstruir manualmente onde o trabalho está.
- As tarefas chegam órfãs no board, sem vínculo com o requisito que as originou — perde-se o “porquê”.

A proposta

- **A spec é a fonte de verdade.** Requisitos descritos com cenários de aceite no formato WHEN/THEN.
- **Rastreabilidade.** Cada requisito gera tarefas vinculadas a ele; toda tarefa aponta para a sua origem.
- **Um só lugar.** Discutir, planejar e acompanhar a execução na mesma superfície (o Control Panel).

Como vai funcionar — o ciclo

1. Discutir e versionar a spec no Control Panel, marcando o que é novo (ADDED) e o que é alteração (MODIFIED).
2. A cada versão aprovada, a spec é materializada no repositório (commit) — o PR carrega spec e código no mesmo diff.
3. As tarefas, vinculadas aos requisitos, recebem um responsável: uma pessoa ou um agente de IA.
4. Execução: o agente abre uma sessão no harness escolhido com o briefing injetado via MCP; o humano trabalha no IDE com branch/PR.
5. Validação Pass 5 (Pullwise) confere o código contra os cenários de aceite do requisito.

- 6. O “onde estamos” fica sempre coerente, pois é derivado dos eventos — não digitado à mão.

Modelo de status

O Pullwise no fluxo justifica dois estados além do óbvio: a validação e a reprovação.

Status	Significado
A fazer	Tarefa ainda não iniciada.
Em execução	Dev ou agente trabalhando na tarefa.
Em validação	PR aberto; o Pass 5 (Pullwise) está conferindo o código contra a spec.
Concluída	Aprovada no Pass 5 — o único caminho para “pronto”.
Bloqueada	Impedida de prosseguir, sempre com motivo registrado.
Ajustes necessários	Reprovada no Pass 5; a crítica volta para a execução.

Regra de ouro: o estado “concluída” nunca é definido pelo agente nem pelo dev — ele é atribuído pelo Pass 5. É isso que impede o descolamento entre spec e código.

Humanos e agentes, juntos

- **Responsável por tarefa.** Pessoa (com foto) ou agente de IA (com ícone) — dá para ver e filtrar por quem executa.
- **Duas pistas de execução.** Humano (IDE + commits/PR) e agente (sessão no harness, status reportado via MCP).
- **Harness plugável.** Claude Code, Codex, Gemini CLI, Cursor — todos falam o mesmo contrato MCP com o workspace. Suportar mais um harness é configuração, não reescrita.
- **Modelo LLM à escolha.** O usuário define qual modelo cada harness usa. O harness é a ferramenta; o modelo é o cérebro.

Da spec aos testes

Os cenários de aceite já são, na prática, especificações de teste. A partir do spec.md, o Control Panel gera um esqueleto de testes (JUnit 5): um teste por cenário, rastreável.

- Cada cenário WHEN/THEN vira um método de teste nomeado que cita o requisito de origem.
- **O Pass 5 exige cobertura:** cenário sem teste correspondente reprova. A spec sempre manda; os testes são derivados dela, nunca escritos em paralelo.

Onde a documentação da spec vive

- **Versionada no Control Panel.** Histórico semântico por requisito — quem mudou o quê, e quando.

- **Materializada no repo.** A cada aprovação, os arquivos da spec são commitados; o agente e o PR sempre veem a versão vigente.
- **Sem drift e sem lock-in.** Uma única fonte escreve nas duas pontas, e a spec continua sendo markdown que o time possui.

O que muda no seu dia a dia

- Menos troca de ferramenta: a tarefa já chega com o contexto (o briefing do requisito e seus cenários).
- **Convenção simples:** commit/branch referenciam o id da tarefa; o agente reporta o status via MCP conforme conclui cada item.
- Toda tarefa aponta para o requisito de origem — o “porquê” está sempre a um clique.
- As restrições do projeto (ex.: PostgreSQL sem prefixo de tabela) viajam no briefing para humanos e agentes.

Para discutir com a equipe

- Confirmar a granularidade do vínculo tarefa ↔ código (proposta: por tarefa atômica, com o agente reportando via MCP).
- Quais harnesses e modelos adotamos primeiro.
- Convenção de testes e a meta de cobertura cenário ↔ teste exigida pelo Pass 5.
- O que o Pass 5 deve reprovar de fato — critérios e severidade.